Using GPU Parallelization to Solve Heterogeneous-Agent Models with Aggregate Shocks

Robert Kirkby

Victoria University of Wellington
robertdkirkby.com
vfitoolkit.com

July 6, 2026

Overview

- Hanbaek Lee's Matched-Expectations Path algorithm is a global non-linear solution method.
  Lee - A Global Dynamic Nonlinear Solution Framework and the Repeated Transition Method.
  'Repeated Transition Path' is already a toolkit feature, so I renamed Lee's algorithm.
- Solves Heterogeneous Agent (incomplete markets) models in the Sequence-Space.
- Here I show how to speed it up >25x using GPUs.

Standard Transition Path

- First, say we have a model without aggregate shocks, e.g., Aiyagari 1994 model.
- We can solve the transition in the model to a final stationary eqm.
- Standard algorithm is,
  1. Solve the agent value function problem (e.g., by backward iteration with value function iteration).
  2. Simulate the agent distribution (e.g., by forward-iteration, or by monte-carlo simulation of agents).
  3. Evaluate model statistics.
  4. Evaluate general equilibrium conditions.
  5. Update the path on general equilibrium 'prices'.
  6. Repeat steps 1-5 until the path on prices converges.

Matched-Expectations Path

- Now, for a model with aggregate shocks.
- In the sequence-space this is a 'generalized transition path'.
  The aggregate shocks become just like an exogenous path parameter (like the tax rates in a transition path to model tax reforms).
- Challenge is that the expectation for next period is very complex.
- The insight of Hanbaek Lee: We can construct the expectations term by 'matching' (next) periods.

Matched-Expectations Path: How to Match? Toy Example.

- Simple example of 'matching' without agent distribution.
- Say we have binary-valued aggregate shock S which takes values S^1 and S^2.
- We want to calculate value function $V(S)$.
  Obviously you should do this with standard value fn iteration, but we will see 'path' approach.
- We simulate 'path' $\{S^1, S^1, S^2, S^1, S^2, S^2, ..., S^1\}$.
- We have some initial guess for $V_t(S)$ at each period.
  E.g. solve model replacing shocks with the mean shock, just use this solution for all periods.
- To be able to do backward-iteration of the value fn on this path, we need 'expected next period value fn', $E[p_1 V_{t+1}(S^1) + p_2 V_{t+1}(S^2)]$.
- But we just have 'next period' for a single aggregate shock value.
  E.g. in period 3 of my simulation, we only have $V_{t+1}(S^1)$ next period, but missing $V_{t+1}(S^2)$ next period.
- **Match the Expectations**: Just use any other $V_\tau(S^2)$ in place of $V_{t+1}(S^2)$; we know that state-space is same in both, so V must be same in both (in final solution).
- Once we create the expected next period value fns for each t in this way, simply solve by backward iteration of value function iteration.

Matched-Expectations Path: How to Match?

- Matching is based on the 'same state-space, other values of the aggregate shocks'.
  My toy example didn't have to deal with 'same state-space'.
- State-space now includes the agent distribution, so in matching we are looking for 'same agent distribution, different aggregate shocks'.
- In practice, 'same agent distribution' will often be done as 'same means of endogenous and exogenous states'.
  Rather than, e.g., Kolmogorov-Smirnov distance.
- Lee (2026) shows that aggregate capital is 'sufficient statistic' for the matching in Krusell-Smith 1998 model. We are still learning what 'matching' should involve in more complex models.

In model where idiosyncratic exogenous shocks are independent of aggregate state they can be ignored. When they depend on aggregate shock (this period and/or next period agg shock) they need to be part of matching. In KS1998 model, the calibration means the aggregate shock is a sufficient statistic for the idiosyncratic shock distribution.
Alternative interpretation: simple matching on mean/variance/etc is a form of 'bounded rationality'.

Matched-Expectations Path: The algorithm

- Say we have a model with aggregate shocks, e.g., Krusell-Smith 1998 model.
- Write sequence-space as a 'generalized transition path'.
  Simulate a sequence for aggregate shocks, put an initial guess for value fn each period.
- Matched Expectations Path algorithm is,
  0. Match the Expectations: Create next period expected value fn for each period.
  1. Solve the agent value function problem (e.g., by backward iteration with value function iteration).
  2. Simulate the agent distribution (e.g., by forward-iteration, or by monte-carlo simulation of agents).
  3. Evaluate model statistics.
  4. Evaluate general equilibrium conditions.
  5. Update the path on general equilibrium 'prices'.
  6. Repeat steps 0-5 until the path on prices converges.

Matched-Expectations Path: Advantages

- Why Matched-Expectations Path is so powerful?
- Everything we know about solving transition paths can be used here.
- So we know how to solve
  - Markov shocks, iid shocks, Semi-exo shocks
  - Multiple endogenous states, including risky assets, human capital, etc.
  - Multiple aggregate shocks.
  - Epstein-Zin preferences, Quasi-Hyperbolic discounting, Prospect theory, etc.
- Global and non-linear! So you can study fiscal policy ;) (Bianchi and Kaplan, 2026)

Matched-Expectations Path: Adapt to GPU

- While we know how to solve them, curse of dimensionality has not gone anywhere.
- So we need to sweat the details to make the implementation fast.
- Brings us to the contribution of this paper.
- Use GPUs.
- How to GPU: *parallelize everything, try to avoid running out of memory*.

Matched-Expectations Path: Adapt to GPU

- Matched Expectations Path algorithm on GPU is,

  Simulate a sequence for aggregate shocks, put an initial guess for value fn each period.

  0. Matched Expectations: Create next period expected value fn for each period.
  1. Solve the agent value function problem in parallel!.
  2. Simulate the agent distribution (e.g., by forward-iteration, or by monte-carlo simulation of agents).
  3. Evaluate model statistics.
  4. Evaluate general equilibrium conditions.
  5. Update the path on general equilibrium 'prices'.
  6. Repeat steps 0-5 until the path on prices converges.

- Key observation: after matching, we can solve value fn in parallel, no need to backward-iterate!

  Normally, we have to solve $t+1$ to get expected value fn so we can solve t. So normally we have to backward-iterate. But matching step eliminates this dependence.

Matched-Expectations Path: Adapt to GPU

- We can make some other improvements.

  Simulate a sequence for aggregate shocks, put an initial guess for value fn each period.

  0. Matched Expectations: Create next period expected value fn for each period in parallel.
  1. Solve the agent value function problem in parallel.
  2. Iterate the agent distribution using Tan improvement.
  3. Evaluate model statistics in parallel.
  4. Evaluate general equilibrium conditions.
  5. Update the path on general equilibrium 'prices'.
  6. Repeat steps 0-5 until the path on prices converges.

- Tan improvement to iterate agent distribution (Tan, 2020).

  Step 2 is memory intense so leave on CPU, Tan improvement is so good it is fast anyway. Iterate the path rather than t (will explain this later).

- Parallelize Step 0 and Step 3 (Ocampo and Robinson, 2023).

  Matching distances for the idiosyncratic exogenous states can actually be precomputed and then reused every iteration. Steps 5 and 6 are just simple matrix operations, so on GPU are going to naturally be done in parallel without thinking about them.

Matched-Expectations Path: Adapt to GPU

- Value fn iteration in parallel:
- In Krusell-Smith model we will have,
  - 501 points on next period assets.
  - 501 points on this period assets.
  - 2 idiosyncratic shock values.
  - 1000 time periods in path.
- Return function is already a roughly 4gb matrix!
  Double floating point. So if we put a further 50 points on labor we would be out of memory already.
- Need some tricks to avoid running out of memory, and to give even more speed.

Matched-Expectations Path: Adapt to GPU

- Divide-and-Conquer: exploit monotonicity to reduce number of next period asset points consider.
  - reduces memory, improves speed
  - Gordon and Qiu (2018) shows how in serial
  - I adapt to parallel GPU version.
- Interpolation of next period endogenous state.
  - slower and more memory...
  - ...but can use a smaller grid to start, so faster and less memory
  - Standard approaches are all serial
  - I adapt to parallel GPU version called 'grid interpolation layer'.

- Important: reduces number of grid points considered, releasing memory bottleneck.
  GPUs have way less memory than CPUs.

Matched-Expectations Path: In Practice

- All of this becomes a one-line command in VFI Toolkit:

  `RecursiveGeneralEqmWithAggShocks_InfHorz()`

- Time to do four examples!
- All examples are running on my *desktop* with Nvidia RTX 4000 Ada GPU (20gb GDDR).

HA-RBC Model 1: Krusell-Smith 1998

- Household problem:
  $V(a, z, S, \mu) = \max_{a'} u(c) + E[V(a', z', S', \mu')]$
  $s.t.\ \ c + a' = w\,z + b(1 - z) + (1 + r)a$
- Representative Firm with Cobb-Douglas Prodn fn: so general eqm eqn is: $r = \alpha S K^{\alpha-1} L^{1-\alpha} - \delta$.
- z idiosyncratic shock, two values: employed / unemployed.
- S aggregate TFP shock, two values: boom / recession.
  Idiosyncratic transitions depend jointly on current and next aggregate state.
  Calibration means aggregate unemployment takes just two values, boom and recession.
- Matching distance: $|K_t - K_\tau|$.
  Lee (2026) shows aggregate capital is a sufficient statistic for the matching in KS1998.
  S' is actually a sufficent statistic for distribution of z in the KS1998 calibration.
- 501 asset grid points, $n\_S = 2$, $n\_z = 2$, $T = 1000$.
  vfoptions.ngridinterp=50

I have written the household problem in state-space notation, μ is the agent distribution.

Figures

HA-RBC Model 1: Comparison to other KS1998 solutions

- DeepHam solves Krusell-Smith in 59 minutes on GPU (Han et al. (2026), Table 3).
- DeepHam+Finite Difference solves Krusell-Smith in 10 minutes on GPU (Abbott and Phan, 2026).
- Matched Expectations Path is 14 minutes on CPU (Lee (2025), runtimes by email).

  Before this Conference

  Before this Conference
- Matched Expectations Path is 25 *seconds* on GPU

GPU times are on Nvidia A100, except mine on RTX 4000 Ada. But both are essentially same GPU compute. Linear solutions with SSJs are sub 1 second (Table 1 of Auclert et al. (2021)).

HA-RBC Model 1: Comparison to other KS1998 solutions

- DeepHam solves Krusell-Smith in 59 minutes on GPU (Han et al. (2026), Table 3).
- DeepHam+Finite Difference solves Krusell-Smith in 10 minutes on GPU (Abbott and Phan, 2026).
- Matched Expectations Path is 14 minutes on CPU (Lee (2025), runtimes by email).
- Deep Learning in the Sequence Space is 1 minute on GPU (Azinovic-Yang and Žemlíčka, 2025)
- Structural Reinforcement Learning is 37 *seconds* on GPU (Yang, Wang, Schaab, and Moll, 2025)
- Matched Expectations Path is 25 *seconds* on GPU

GPU times are on Nvidia A100, except mine on RTX 4000 Ada. But both are essentially same GPU compute. Linear solutions with SSJs are sub 1 second (Table 1 of Auclert et al. (2021)).

HA-RBC Model 1: Krusell-Smith 1998

- Matched Expectations Path is 25 *seconds* on GPU (varies from 22 to 28s).
- 10 seconds are just solving the model without aggregate shocks, which could be sped up.
  Improve initial guess for r, and use shooting algorithm on r. I haven't bothered as is fast enough anyway for my purposes.
- 15 seconds iterating on the generalized transition path.
- ~~Fast enough for Bayesian estimation of the global non-linear model!~~
  ~~Kalman filter is linear, so no use. Needs particle filter or similar.~~

Figures

HA-RBC Model 2: Uncertainty Shocks

- Household problem:
  $V(a, z, S_{TFP}, S_{\text{Uncert}}, \mu) = \max_{a'} u(c) + E[V(a', z', S'_{TFP}, S'_{\text{Uncert}}, \mu')]$
  $s.t.\ c + a' = w\,z + (1 + r)a$
- Representative Firm with Cobb-Douglas Prodn fn: so general eqm eqn is: $r = \alpha S_{TFP} K^{\alpha-1} L^{1-\alpha} - \delta$.
- Two aggregate shocks: (log) TFP S_{TFP} and (log) uncertainty S_{Uncert}, both AR(1).
- Idiosyncratic shock: (log) effective labor units z is AR(1).
- S_{Uncert} scales the innovation std of both S_{TFP} and z.
  So high uncertainty raises aggregate volatility *and* idiosyncratic volatility.
- Demonstrates: multiple correlated aggregate shocks, notice that to cover all combos we need bigger T.
  Notice how curse-of-dimensionality is not gone, more aggregate shock values means bigger T is needed for 'good' matches.
- 601 asset grid, $n\_S = [5, 2]$, $n\_z = 4$, $T = 10000$. Runtime is 9 minutes.
  Use *recursiveeqmoptions.divideT* $= 15$ to split T into 15 parts, to reduce memory use at cost of slower codes. Otherwise this model runs out of memory.

Figure

HA-RBC Model 3: Entrepreneurial Choice

- Entrepreneurial-Choice model of Kitao (2008), add aggregate shocks to productivity.
- Household problem:
  $V(a, e, \eta, \theta, S, \mu) = \max_{e,a'} u(c) + E[V(a', e', \eta', \theta', S', \mu')]$
  s.t.
  if e=0 (worker): $c + a' = w\,\eta + (1 + r)a$
  if e=1 (entrepreneur): $c + a' = EntrepreneurialIncome(k, n, \theta)$
- Representative Firm with Cobb-Douglas Prodn fn: so general eqm eqn is: $r = \alpha S K_{Corp}^{\alpha-1} L_{Corp}^{1-\alpha} - \delta$.
  Model also has taxes and gov spending that I omit here; there is a government budget general eqm condition.
- Two endogenous states: assets, a, and entrepreneur/worker e.
- Two idiosyncratic shocks: effective labor units η, Entrepreneurial productivity θ.
  One aggregate shock: TFP S, same binary valued S as in KS1998.
- Demonstrates: two endogenous states and two exogenous states household, match on mean of each.
- 1001 asset grid, $n\_S = 2$, $n\_z = [5, 4]$, $T = 1000$. Runtime is 40 minutes.
  Use *recursiveeqmoptions.divideT* $= 40$ to split T into 40 parts, to reduce memory use at cost of slower codes. Otherwise this model runs out of memory.

HA-RBC Model 4: KS1998 with Quasi-Hyperbolic Discounting

- Same KS1998 model as HA-RBC Model 1, but replace exponential discounting with quasi-hyperbolic (β_0,β).
- Present-bias parameter β_0 on next-period continuation value, alongside standard long-run discount factor β.
- Solve both variants:
  - **Naive**: current self is present-biased but believes future selves are exponential discounters.
  - **Sophisticated**: current self anticipates that future selves will also be present-biased.
- Demonstrates: non-standard preferences are easy to plug in.
- Same grids as KS1998, each of Naive and Sophisticated has almost the same runtime as basic KS1998.

Matched Expecations Path

- Summarizing the Pros and Cons:
- Easy to use. Can already solve lots of cool models!
- Main weakness is currently stability of algorithm, I often struggle with convergence when adding endogenous labor.
- I expect this is about finding better approaches to matching and price path update. Also appears pretty sensitive to getting accurate grids. I don't think this is anything deep, and expect it to be resolved in near future. Already making good progress on this.

Matched Expecations Path

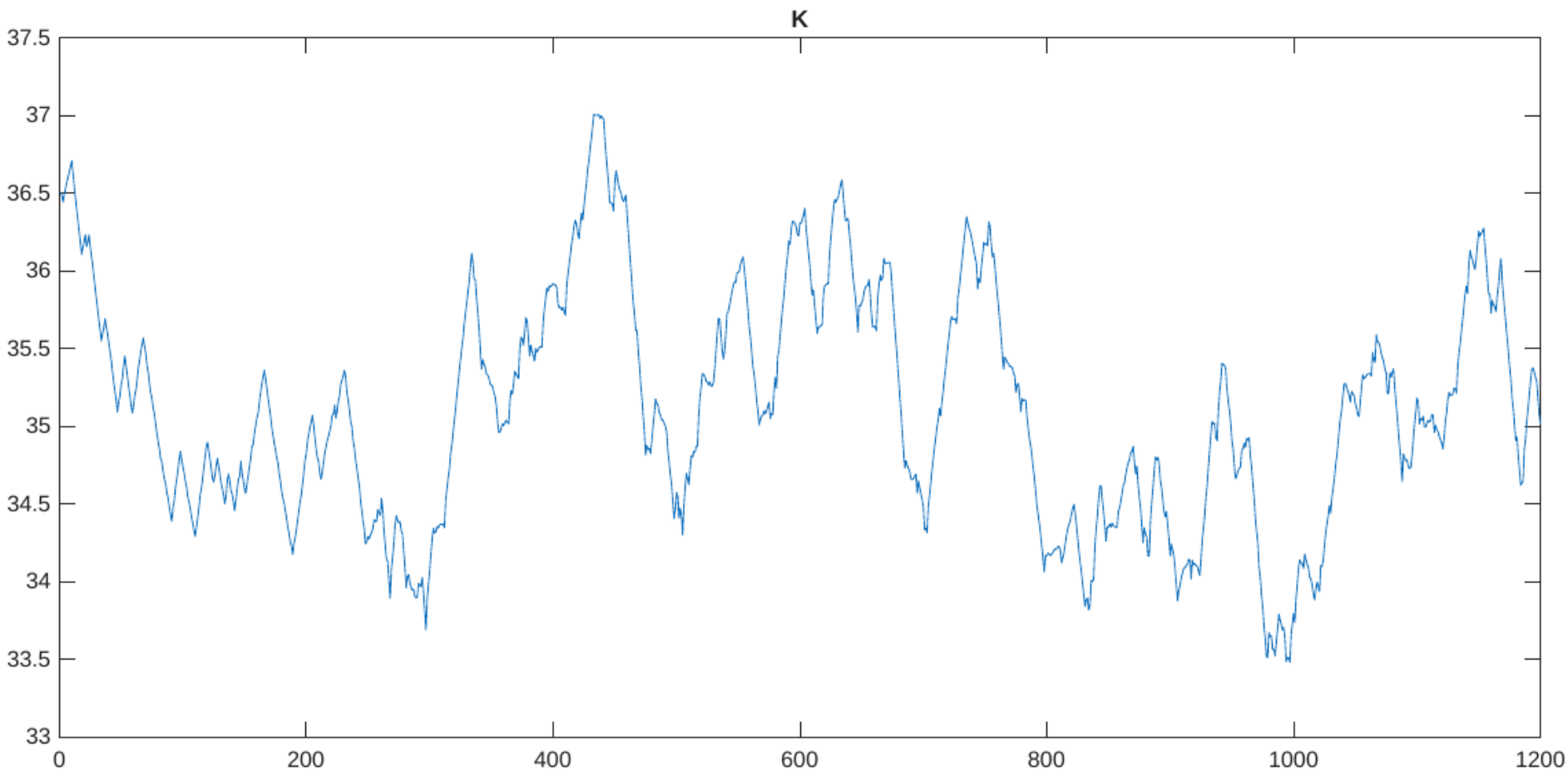
- Lot's of open questions on implementation details that will lead to further improvements:

- How to match? What metric? And how to code for speed?
  I precompute match distances for exo states. Currently I use a mask on *Sprime*, then compute all distances, but maybe sort first? Or with multiple endo states, partial match on first dim then only measure some on second dim?
- Replace simulation of aggregate shock with quasi-MC.
  qMC of an MC, not to be confused with MCqMC. Should we think about agent dist while doing this? Can help reduce the size of T we need.
- Initial guess for value fn and agent dist path?
  Currently just use stationary eqm of model without aggregate shocks. Maybe use SSJ to build initial guess?
- How to update agent dist? Iterate t, or all in one?
  Do I create $\mu_{i,t}$ from $\mu_{i-1,t-1}$ (last iteration) or $\mu_{i,t-1}$ (last period). I do last iteration, Lee does last period.
- How to update price path? Faster? More stable?
  Currently just basic shooting algorithm. Plenty of room for improvement.
- Output is the 'Generalized Transition Path', need lots of functions to make analysing it easy.

Matched Expecations Path

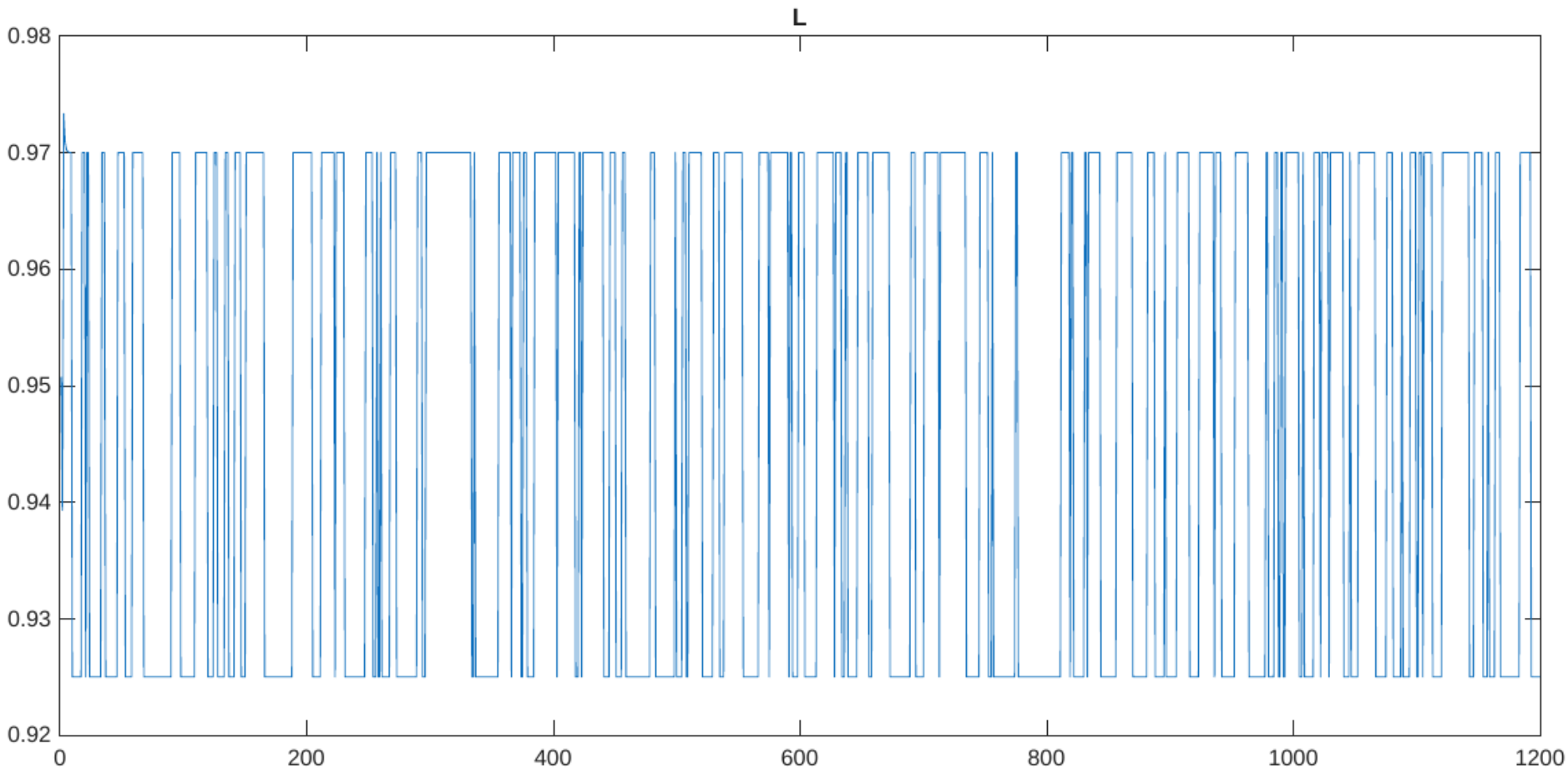
- Hanbaek Lee's Matched Expectations Path opens the doors to solving all sorts of fun stuff!
- GPU makes it fast.
- VFI Toolkit makes it easy.
- Codes for the four examples, see bottom of robertdkirkby.com

HA-RBC Model 1: Figure 2A



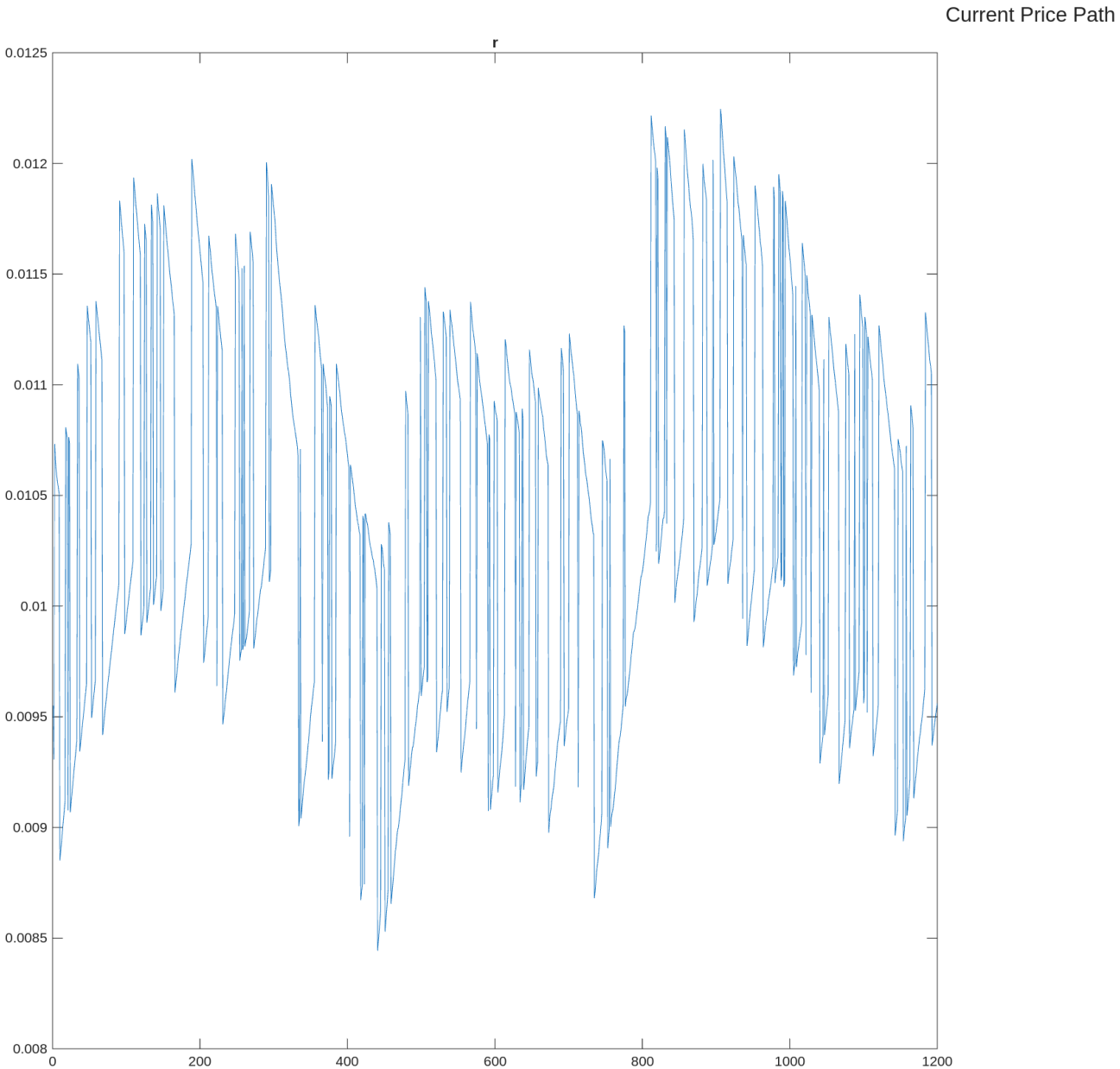
Back 1 Back 2

HA-RBC Model 1: Figure 2B

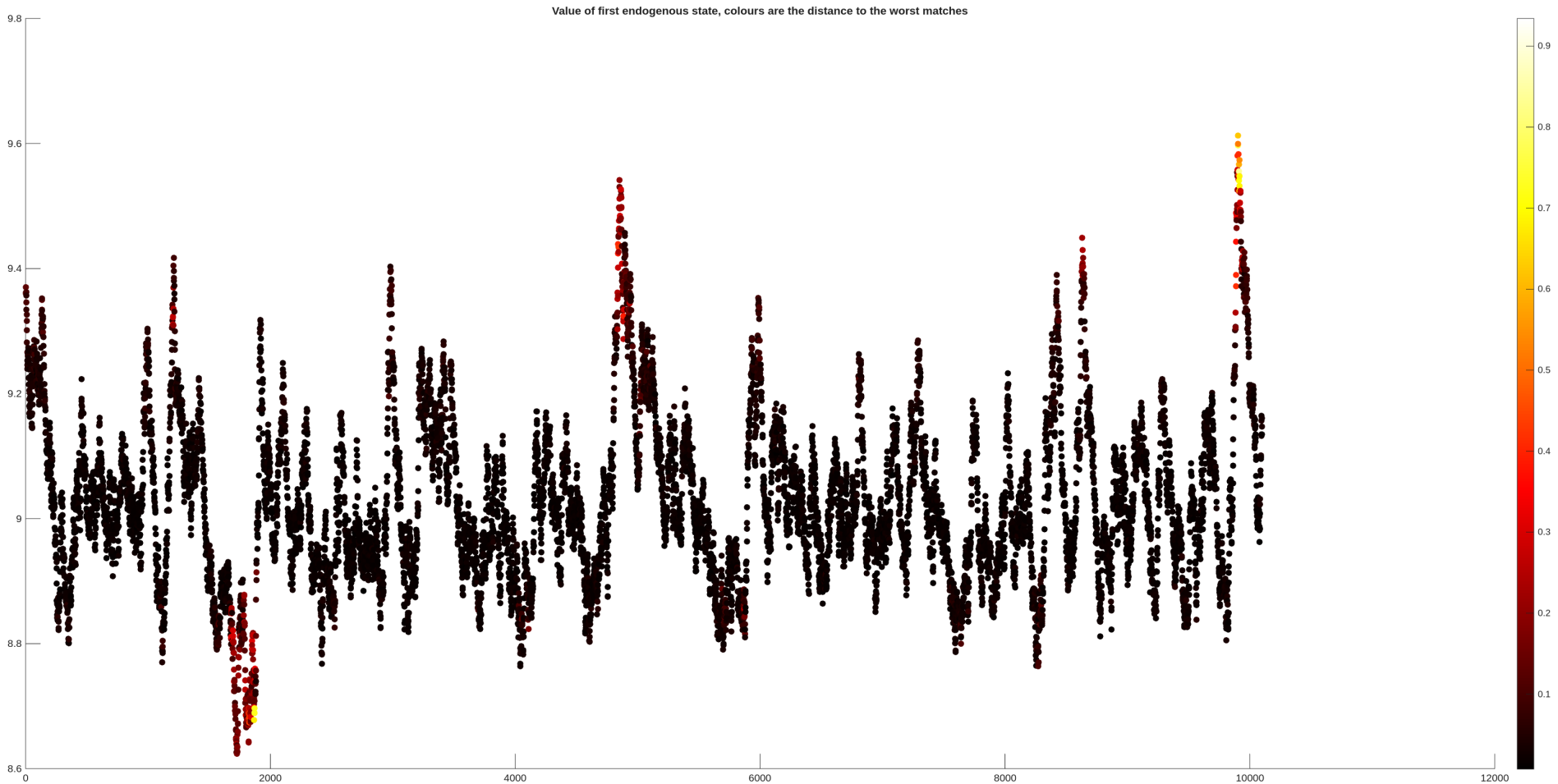


Back 1 Back 2

HA-RBC Model 1: Figure 1



HA-RBC Model 2: Figure 5



Back

References I

Brant Abbott and Nam Van Phan. Using deep learning and finite-difference methods together to solve heterogeneous agent models with aggregate shocks. *Working Paper*, 2026.

Adrien Auclert, Bence Bardoczy, Matthew Rognlie, and Ludwig Straub. Using the sequence-space jacobian to solve and estimate heterogeneous-agent models. *Econometrica*, 4(2), 2021. doi: https://doi.org/10.3982/ECTA17434.

Marlon Azinovic-Yang and Jan Žemlíčka. Deep learning in the sequence space. *CERGE-EI Working Paper*, No. 802, 2025. arXiv:2509.13623.

Javier Bianchi and Greg Kaplan. How small is small? non-linearities in heterogeneous agent models. Working paper, available at http://javierbianchi.com/, 2026.

Grey Gordon and Shi Qiu. A divide and conquer algorithm for exploiting policy function monotonicity. *Quantitative Economics*, 9(2), 2018. doi: https://doi.org/10.3982/QE640.

Jiequn Han, Yucheng Yang, and Weinan E. Deepham: A global solution method for heterogeneous agent models with aggregate shocks. *Quantitative Economics*, 2026. doi: https://doi.org/10.3982/QE2190.

References II

Sagiri Kitao. Entrepreneurship, taxation and capital investment. *Review of Economic Dynamics*, 2008. doi: https://doi.org/10.1016/j.red.2007.05.002.

Hanbaek Lee. Global nonlinear solutions in sequence space and the generalized transition function. Working paper, 2025.

Hanbaek Lee. Dancing on the saddles: A geometric framework for stochastic equilibrium dynamics. Working paper, 2026.

Sergio Ocampo and Baxter Robinson. Computing longitudinal moments for heterogeneous agent models. *Computational Economics*, 2023. doi: https://doi.org/10.1007/s10614-023-10493-1.

Eugene Tan. A fast and low computational memory algorithm for non-stochastic simulations in heterogeneous agent models. *Economics Letters*, 193, 2020. doi: https://doi.org/10.1016/j.econlet.2020.109285.

Yucheng Yang, Chiyuan Wang, Andreas Schaab, and Benjamin Moll. Structural reinforcement learning for heterogeneous agent macroeconomics. *Swiss Finance Institute Research Paper*, 2025. arXiv:2512.18892.